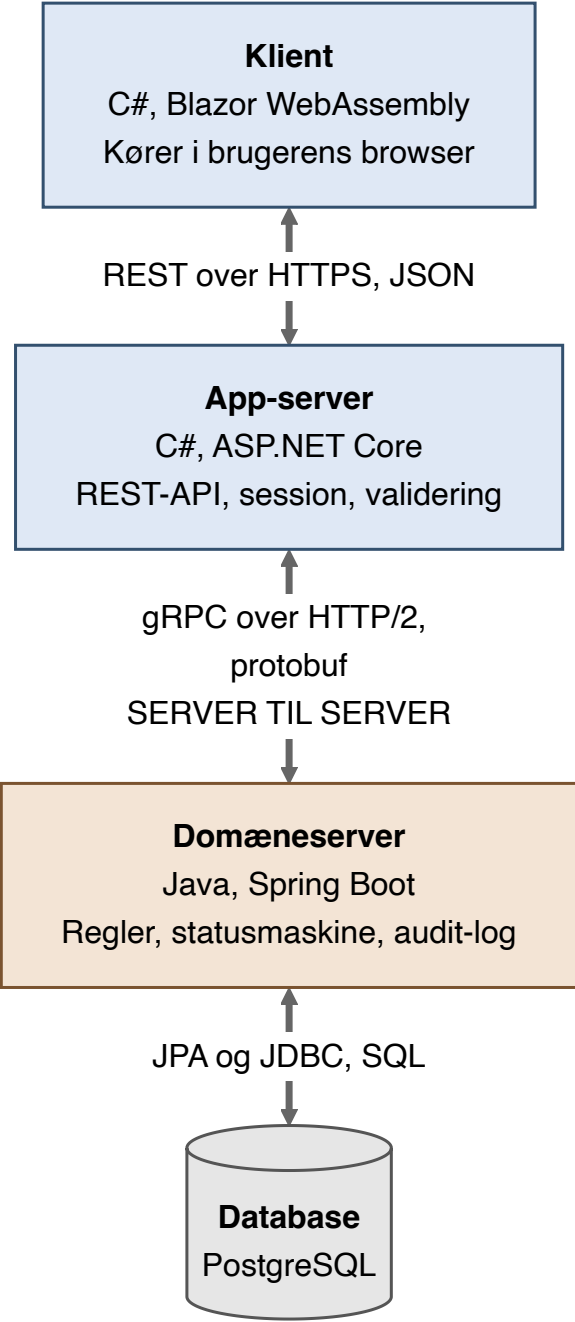


Systemarkitektur

Leveringsplanlægger · SEP3, VIA University College, 3. semester, efterår 2026
Abdirahman Isse Ibrahim Mahamed, studienummer 364963, TS-SEP3-A26 · Virksomhed: ZNG
Del 1 af 3 · 28. september 2026



Hvordan kommunikationen realiseres, og hvilke teknologier der bruges hvor

Forbindelse	Teknologi	Sådan realiseres den, og hvorfor netop den
Klient til app-server Browser mod C#	REST over HTTPS, data som JSON	Klienten er en selvstændig proces i browseren og kalder navngivne ressourcer med GET, POST, PUT og DELETE. Svarene er JSON. REST er valgt, fordi kaldene er få og faste, og fordi browseren taler HTTP i forvejen uden ekstra bibliotek. Sessionen bæres af en cookie, der signeres med HMAC af app-serveren.

App-server til domæneserver C# mod Java <i>server til server</i>	gRPC over HTTP/2, data som protobuf	Kontrakten skrives først i en .proto-fil, og klientkode til C# og serverkode til Java genereres ud fra den samme fil. De to sprog kan derfor ikke blive uenige om formatet. Kaldene er synkrone forespørgsler med svar, og protobuf er binært og fylder mindre end JSON. Domæneserverens port er kun åben på det interne netværk, så al adgang til data går gennem app-serveren.
Domæneserver til database	JPA over JDBC, SQL	Kun domæneserveren rører databasen. JPA giver interaktive transaktioner, så en statusændring, en lagerkontrol og en linje i audit-loggen skrives i samme transaktion. Optimistisk låsning håndhæves med en versionskolonne på ordren.

Server til server, og hvorfor systemet er delt i to

- **App-serveren** ejer det brugervendte: login, sessioner, rate limit på login, validering af input og de tal, dashboardet viser. Den ejer ingen forretningsregler.
- **Domæneserveren** ejer databasen og alle regler: hvilke statusskift der er lovlige, hvem der må udføre dem, om der er varer nok på lageret, og hvad der skrives i audit-loggen. Reglerne findes kun her.
- Fordi domæneserveren ikke er tilgængelig udefra, kan reglerne ikke omgås ved at kalde uden om brugerfladen. Det er kravet om **server til server** opfyldt med en begrundelse, ikke bare et ekstra led.

To netværksteknologier. Kravet er opfyldt af REST mellem browser og C#, og gRPC mellem C# og Java. SignalR indgik i projektbeskrivelsen som et tillæg til live-opdatering, men er taget ud efter vejleders anbefaling den 26. september. Dashboardet opdaterer sig i stedet ved at hente ændringer med faste mellemrum over REST.

Hvad hver del er bygget af

Del	Teknologi	Ansvar
Klient	C#, Blazor WebAssembly	To visninger i én app: dashboard til disponenten på desktop og en visning til chaufføren på telefon, hvor et statusskift kræver højst to tryk
App-server	C#, ASP.NET Core	REST-kontrakt, session-cookie med HMAC, rate limit på login, validering, aggregering af dashboard-tal, gRPC-klient
Domæneserver	Java, Spring Boot, gRPC, JPA	Statusmaskine med syv tilstande, ejerskabstjek, lagerregler, audit-log, transaktioner og optimistisk låsning
Database	PostgreSQL	Kunder, lagre, varer, beholdning, ordrer, varelinjer, brugere, sessioner og hændelser